

**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory Work No. 3

Operating Systems

The Command Line: Shell, Pipes and Text Filters

Student: Bianca-Andreea Tataroi
Group: FAF-242

Checked by: Reițman Patricia

Chișinău – 2026

Contents

1 Laboratory Work No. 3 **2**
1.1 The Command Line: Shell, Pipes and Text Filters 2
1.2 Objective 2
1.3 Part 1 – Move Fast, Find Help 2
1.4 Part 2 – What the Shell Does to a Command Line 3
1.5 Part 3 – Redirection, Chaining and Jobs 4
1.6 Part 4 – One-Line Text Processing 6
1.7 Part 5 – Bash Script 9

2 GitHub Repository **10**

3 Conclusion **10**

1 Laboratory Work No. 3

1.1 The Command Line: Shell, Pipes and Text Filters

1.2 Objective

The objective of this laboratory work was to study the Linux command line and the Bash shell. The work focused on shell built-ins, command history, PATH, shell expansion, quoting, redirection, background processes, pipes, text filters and Bash scripting.

1.3 Part 1 – Move Fast, Find Help

The first part focused on identifying shell built-ins, external commands and the directories contained in the PATH variable.

The following commands were used:

```
type cd
type ls
type echo
which ls
echo $PATH
echo $PATH | tr ':' '\n'
nosuchprog
echo "exit code: $?"
```

The important results were:

```
cd is a shell builtin
ls is aliased to ‘ls --color=auto’
echo is a shell builtin
/usr/bin/ls
exit code: 127
```

The PATH contained 10 printed entries. The program `ls` was located in `/usr/bin`.

The command `cd` must be implemented as a shell built-in because it has to change the working directory of the current shell.

The option used by `ls` to sort files by modification time is:

```
ls -t
```

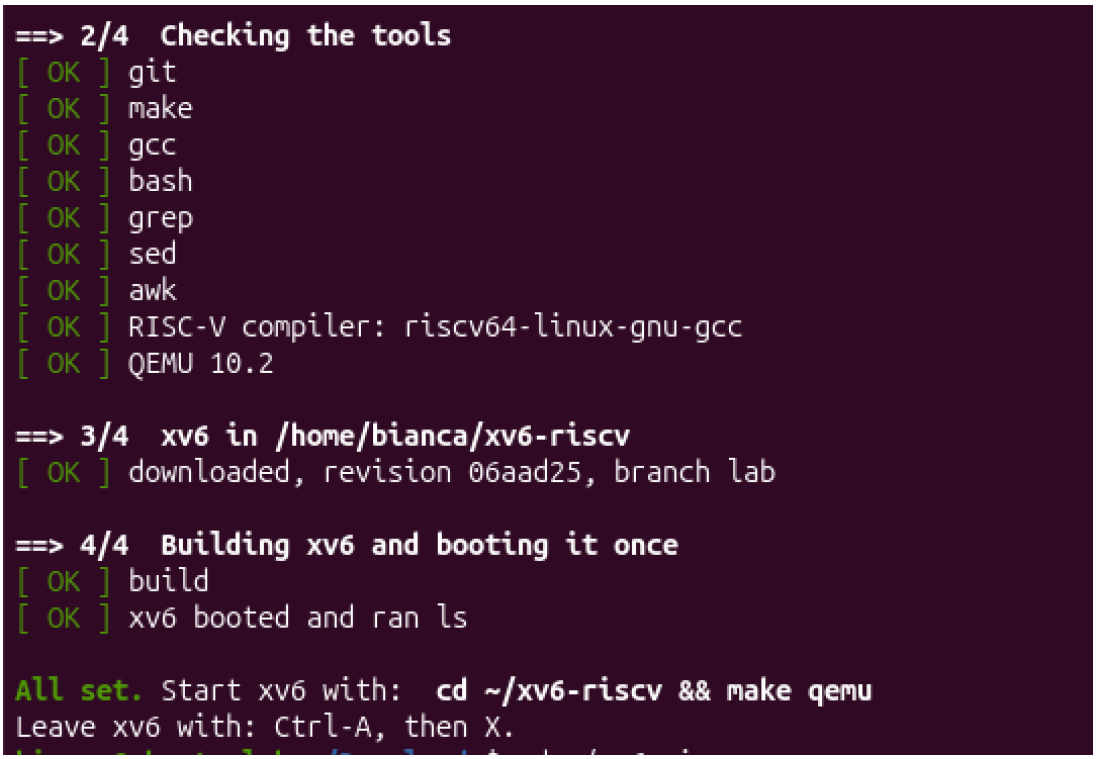


Figure 1: Shell built-ins, PATH and command exit code

1.4 Part 2 – What the Shell Does to a Command Line

Several files were created to test shell expansion:

```
mkdir -p ~/os-lab3/play
cd ~/os-lab3/play
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
```

The following expressions were tested:

```
echo *.c
echo file?.txt
echo [a-b]*
echo *.xyz
ls alfa beta.txt
ls "alfa beta.txt"
echo '$HOME' "$HOME" \$HOME
echo {1..5}
echo $((7 * 6))
```

The results showed that Bash expands wildcard expressions before executing the selected command.

For example:

```
echo *.c
a.c b.c
```

The command:

```
ls alfa beta.txt
```

failed because the shell interpreted the filename as two different arguments.
The correct version was:

```
ls "alfa beta.txt"
```

Quotation marks were also compared:

```
echo '$HOME' "$HOME" \$HOME
```

Single quotes prevent expansion, while double quotes allow variable expansion. A backslash escapes the following special character.

```
bianca@ubuntu-lab:~/os-lab3$ bash oneliners.sh
100
/page0
/page1
/page2
/page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
     14 user6
     14 user4
     14 user2
     14 user1
     14 user0
72
15
10:10 user0
10:15 user5
10:20 user3
     40 /usr/sbin/nologin
      5 /bin/false
      2 /bin/bash
      1 /bin/sync
237
238
```

Figure 2: Shell expansion, quoting and filenames containing spaces

1.5 Part 3 – Redirection, Chaining and Jobs

Standard output and standard error were redirected independently:

```
ls /etc /nope > out.txt 2> err.txt
wc -l out.txt
cat err.txt
```

The obtained output was:

```
236 out.txt
ls: cannot access '/nope': No such file or directory
```

This showed that the normal output was stored in `out.txt`, while the error message was stored in `err.txt`.

Input redirection was also tested:

```
wc -l < access.log
```

Result:

```
500
```

Command chaining was tested using:

```
ls /nope 2> /dev/null || echo "failed with code $?"
```

The result was:

```
failed with code 2
```

The operators have the following meanings:

- `;` – executes commands sequentially;
- `&&` – executes the next command after success;
- `||` – executes the next command after failure.

The order of redirections was compared using:

```
ls /etc /nope > all.txt 2>&1
ls /etc /nope 2>&1 > all.txt
```

The two commands behave differently because Bash processes redirections from left to right.

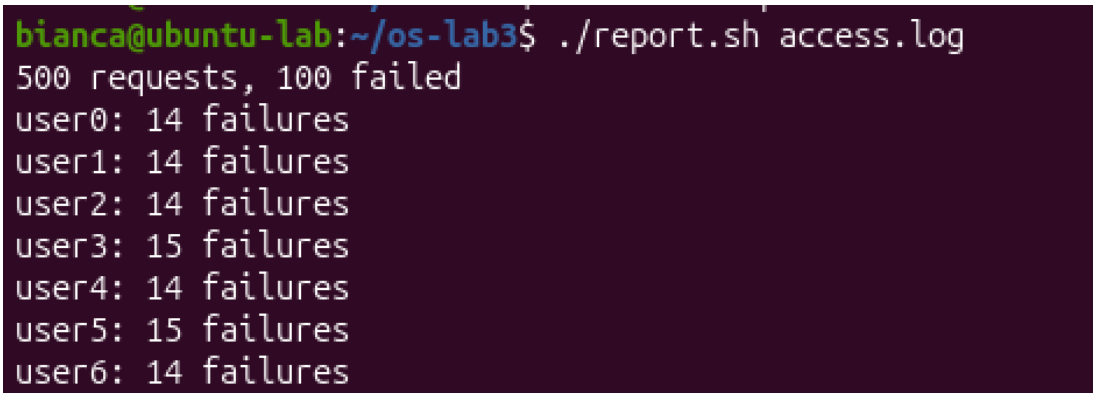


Figure 3: Output redirection and error handling

A background job was also tested:

```
sleep 100 &
jobs
kill %1
jobs
```

The process first appeared as running and was then terminated.

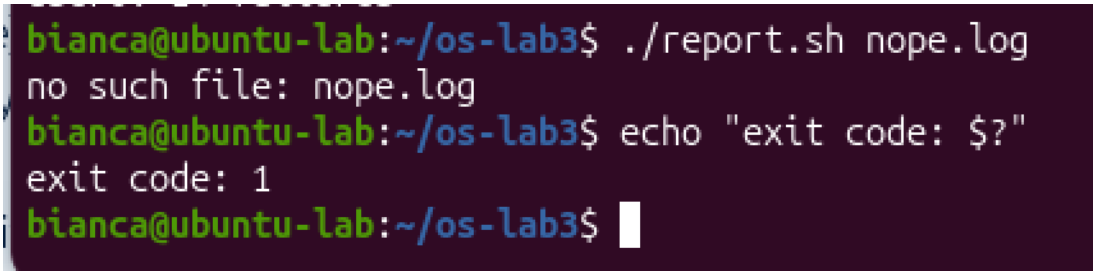


Figure 4: Background process management

1.6 Part 4 – One-Line Text Processing

The file `access.log` contained 500 requests.

The first question counted failed requests:

```
grep -c FAIL access.log
```

Result:

```
100
```

The requested pages were extracted using:

```
cut -d' ' -f5 access.log | sort -u
```

The result contained four pages:

```
/page0
/page1
/page2
/page3
```

The requests per page were:

```
125 /page0
125 /page1
125 /page2
125 /page3
```

The users with the most failed requests were:

```
15 user5
15 user3
```

Therefore, user3 and user5 each generated 15 failures.

For user3, the results were:

```
72
15
```

This means that user3 made 72 total requests and 15 of them failed.
The last three failed requests were:

```
10:10 user0
10:15 user5
10:20 user3
```

The login-shell analysis produced:

```
40 /usr/sbin/nologin
5 /bin/false
2 /bin/bash
1 /bin/sync
```

Finally:

```
ls /etc | wc -l
237

ls -l /etc | wc -l
238
```

The long listing contains one additional line because it displays the `total` value before the directory entries.

```
bianca@ubuntu-lab:~/os-lab3$ cd ~/os-lab3
type cd
type ls
type echo
which ls
echo $PATH | tr ':' '\n'
nosuchprog
echo "exit code: $?"
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
nosuchprog: command not found
exit code: 127
```

Figure 5: Results of the access log analysis

All one-line commands were saved inside the file `oneliners.sh`.

```
bianca@ubuntu-lab:~/os-lab3$ cd ~/os-lab3/play
echo *.c
echo file?.txt
echo [a-b]*
echo *.xyz
ls alfa beta.txt
ls "alfa beta.txt"
echo '$HOME' "$HOME" \$HOME
a.c b.c
file1.txt file2.txt
a.c alfa beta.txt b.c
*.xyz
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
'alfa beta.txt'
$HOME /home/bianca $HOME
```

Figure 6: Execution of oneliners.sh

1.7 Part 5 – Bash Script

A Bash script named `report.sh` was created to automatically analyze the log file.

The script contained the following logic:

```
#!/bin/bash

log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests, $fails failed"

for i in {0..6}; do
    count=$(grep FAIL "$log" | grep -c "user$i")
    echo "user$i: $count failures"
done
```

The file was made executable and then executed using the access log.

The result was:

```
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
```

```
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

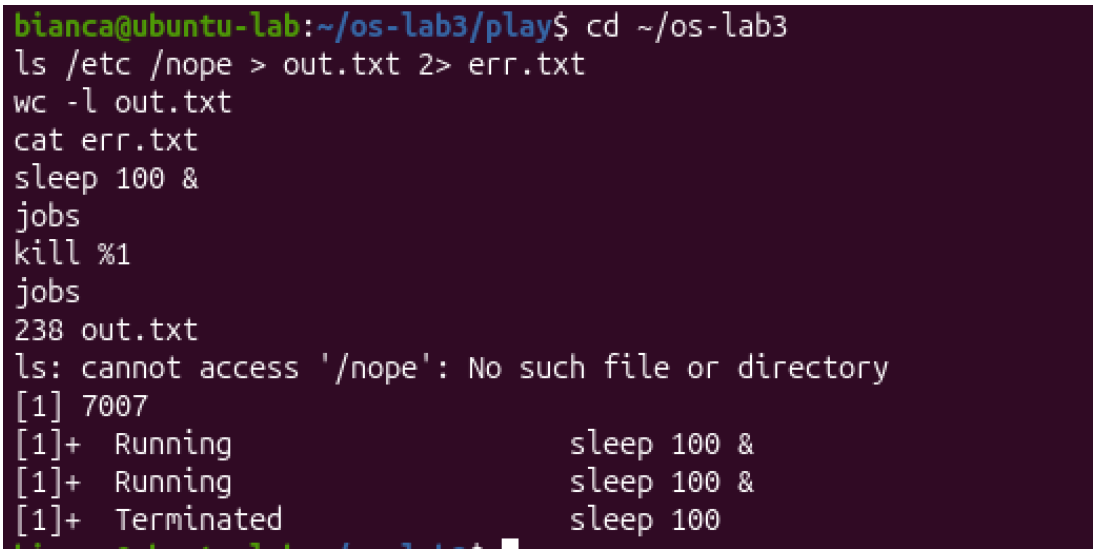


Figure 7: Successful execution of report.sh

The script was also tested using a nonexistent filename.

The output was:

```
no such file: nope.log
```

The script returned exit code 1.

The first line of the script specifies Bash as the interpreter. The `./` prefix is needed because the current directory is not normally included in PATH. The expression `>&2` sends the message to standard error, while `exit 1` indicates failure.

2 GitHub Repository

The source files and the final report for Laboratory Work No. 3 were uploaded to GitHub.

The repository contains the files used for this laboratory work, including the report and the scripts created during the practical tasks.

The repository can be accessed at:

`https://github.com/Biancatataroi/Operating-Systems-Lab3`

3 Conclusion

During this laboratory work, the Linux Bash shell was explored through several practical experiments. Shell built-ins, the PATH environment variable and command exit codes were examined in order to understand how commands are located and executed.

Shell expansion demonstrated that wildcard patterns, variables, quotation marks and arithmetic expressions are processed by Bash before the program itself receives the command-line arguments.

Redirection experiments showed how standard output and standard error can be managed independently. Command chaining and background jobs also demonstrated how Bash controls the execution of processes.

The analysis of the access log demonstrated how small Unix text processing utilities can be combined using pipes in order to solve larger tasks. Finally, the Bash scripts showed how frequently used commands can be automated and how basic error handling can be added.

Overall, this laboratory improved the practical understanding of command-line processing, text filters, shell scripting and process control in Linux.